

53rd SME North American Manufacturing Research Conference (NAMRC 53, 2025)

Assessing the Cybersecurity of Connected 3D Printers Using Large Language Models (LLMs)

Shi Yong Goh^a, Ankush Mishra^a, Manimaran Govindarasu^a,
Baskar Ganapathysubramanian^a, Adarsh Krishnamurthy^{a,*}

^aIowa State University, Ames IA 50014, USA

Abstract

Additive manufacturing (AM) is transforming industries by enabling complex, cost-effective production, yet its integration with Industry 4.0 introduces significant cybersecurity vulnerabilities. This paper examines potential cyber threats in AM workflows, focusing on attacks that manipulate G-code instructions, such as Man-in-the-Middle (MITM) exploits, which could compromise the integrity of critical components. We introduce Large Language Models (LLMs) as a novel tool for detecting malicious alterations in G-code, showcasing their potential for real-time threat detection. Our findings underscore both the risks within AM and the promise of AI-driven cybersecurity solutions to protect evolving intelligent manufacturing systems.

© 2025 The Authors. Published by ELSEVIER Ltd.

This is an open access article under the CC BY-NC-ND license

(<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the NAMRI/SME.

Keywords: Cybersecurity; Large Language Model; Man-in-the-Middle Attack; Connected 3D Printer

1. Introduction

Additive manufacturing (AM) or 3D printing has gained more popularity in recent years for prototyping and small machine parts in multiple industries because of the significant advantages this process can offer. Scanning and rapid prototyping software are expected to have the highest compound annual growth rate (CAGR) of 24% over the period from 2023–2030 [1]. This growth is driven by its ability to reduce manufacturing costs, optimize material wastage, and facilitate design features that cannot be achieved using traditional processes. Relativity Space's Terran 1 rocket is an excellent example of these benefits when it became the first 3D-printed rocket to reach space [2], highlighting the promises of AM for advances in critical systems.

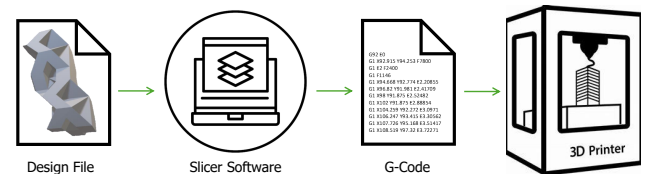


Fig. 1: General workflow for 3D Printing. The design file or the CAD model is sliced into layers by the slicer software and is converted to G-code based on the printer parameters. The G-code file is then sent to the 3D printer for manufacturing.

AM is a critical component of *Industry 4.0*, which integrates automation, data analytics, artificial intelligence (AI), and industrial internet-of-things (IIoT) for efficient, connected production systems [3]. However, this increased connectivity introduces new cyber vulnerabilities throughout the AM workflow, from CAD file creation to final manufacturing, expanding the attack surface for malicious actors. The adoption of AM in critical sectors such as aerospace, healthcare, and automotive industries [4–8] raises

* Corresponding author.

E-mail address: adarsh@iastate.edu (Adarsh Krishnamurthy).

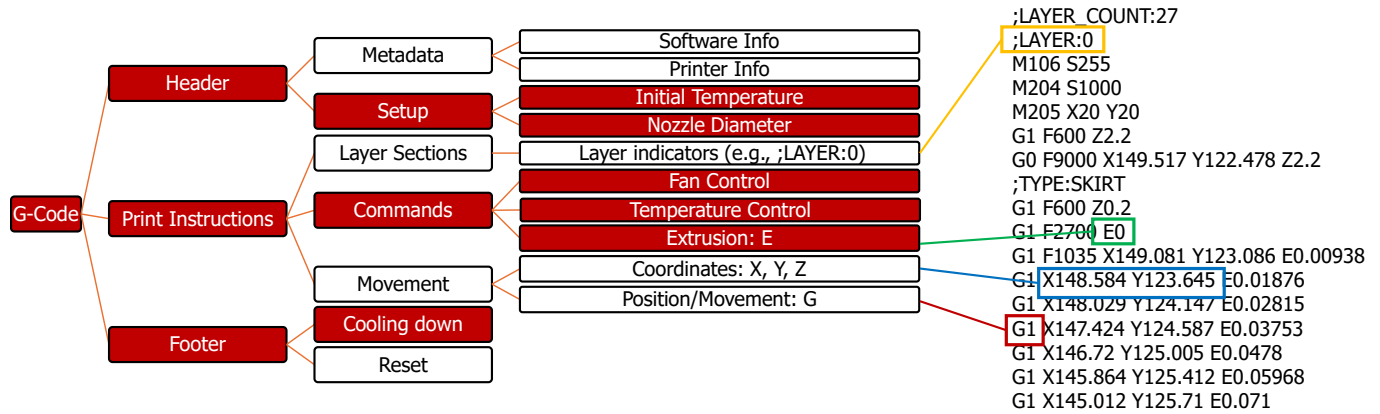


Fig. 2: Structure of the G-Code used to control the 3D printing process.

significant cybersecurity concerns, as compromised parts could have severe consequences. Yampolskiy et al. [9] explored this risk, demonstrating that vulnerable elements in the 3D printing process could be manipulated to produce effects comparable to various weapons.

While AM is recognized as part of IIoT, there are opportunities for more focused research and standardization efforts, specifically addressing the wireless connectivity needs of AM within these broader frameworks. In this paper, we specifically analyze the AM workflow (Fig. 1) and the AM system architecture to identify potential cybersecurity vulnerabilities while considering compatibility with existing infrastructure and security measures that maintain production efficiency. We comprehensively analyze cybersecurity concerns in 3D printing, focusing on potential impacts and mitigation strategies. We examine theoretical vulnerabilities and practical attack scenarios, particularly Man-in-the-Middle (MitM) attacks that can manipulate printing instructions. In these scenarios, we focus specifically on attacks that modify the G-code, which are the instructions sent to the 3D printer to perform the additive manufacturing process.

Finally, we explore Large Language Models (LLMs) as a novel approach for detecting compromised G-code and evaluating their effectiveness in identifying malicious modifications. Our results demonstrate the potential of AI-driven security measures in detecting compromised G-code, contributing to AM security knowledge. Our approach can be effectively used to retrofit existing 3D printers to make them more secure from MitM attacks.

The integration of AI-driven attack detection methods marks a step forward in enhancing the reliability and quality of additive manufacturing processes. Leveraging these advanced techniques enables early detection of potential issues before printing begins, which reduces waste, improves efficiency, and strengthens the overall security and quality of AM products.

2. Background and Related Work

Cybersecurity has become critical to 3D printing systems due to their reliance on big data and cloud technology [10], which enable remote access and real-time monitoring. However, all of these advanced functionality exposes the system to cybersecurity vulnerabilities [11, 12]. Therefore, cybersecurity is essential to safeguard the entire system and protect intellectual property, preventing data theft and unauthorized access to sensitive design files and configuration settings.

Several cyberattack studies [11–16] addressing 3D printer or AM focus on compromising the G-Code file rather than the design file such as STL file or the CAD model file. The primary method for modifying the design file or configuration setting is through the G-code file. When the design file is input to the 3D slicer software, the software slices the model into layers and generates the G-Code file. G-code controls the 3D printer's layer-by-layer build process by instructing the nozzle's movements, adjusting printer settings, and managing other parameters [17]. However, The G-Code file can vary according to different 3D printer firmware or slicer software for specific printers. Typically, the G-Code file is structured into three main sections: the header, the main body, and the footer, as shown in Fig. 2. We used G-code files generated by Ultimaker Cura [18] for this research.

Recent advances in AI and machine learning are creating exciting new possibilities for defect detection and quality control in 3D printing. Machine learning techniques have shown effectiveness in detecting G-code outliers that may lead to printing defects. For example, research has shown that they explored the use of statistical modeling and machine learning to detect subtle malicious edits in G-code files [25], which can analyze the G-code instructions, identifying patterns and anomalies that might be potential issues in the printed part. While these advances are promising, a recent paper [26] highlights machine learning's vulnerability to adversarial attacks, emphasizing the importance of addressing security risks in AI-based detection systems.

Table 1: Critical security vulnerabilities (high and medium severity ratings) in connected 3D printers.

Vulnerability	Description	Base Score CVSS v3	Impacts
Cross-Site Request Forgery (CSRF)	CSRF is an attack that tricks the victim into submitting a malicious request, inheriting the victim's identity and privileges to perform undesired functions on their behalf [19]. This can include actions like changing the victim's email address, password, or initiating unauthorized purchases.	8.8 (High) [20]	Exploitation of this vulnerability could allow attackers to perform actions on the 3D printer's local web server as the authenticated user, leading to unauthorized configuration changes, printing of unauthorized objects, or other malicious actions.
Code Injection	Unsafe handling of 3D model files can allow hackers to run malicious code or inject code into the user-controlled data plane through legitimate data channels.	7.8 (High) [21]	This vulnerability allows attackers to potentially bypass login protections and access unauthorized parts of the system. This can compromise data integrity by enabling unauthorized actions such as modification or deletion of data. Attackers can also inject code to gain control over the system. These actions may go undetected, as attacks carried out through injections are often not logged, reducing the system's visibility and making it difficult to detect the attacks [22].
Clickjacking	A clickjacking vulnerability was identified in the Ultimaker S5 3D printer's local web server, as reported by the National Vulnerability Database of NIST [23]. This vulnerability can be exploited through the printer's settings page.	7.1 (High) [23]	In this scenario, an attacker could craft a malicious web page that uses an iframe to overlay the Ultimaker S5's settings page. When users interact with what appears to be the legitimate settings page, they may unknowingly change printer settings or execute unauthorized actions orchestrated by the attacker.
Cross-Site Scripting (XSS)	XSS vulnerabilities occur when a web application allows malicious scripts to be injected into web pages viewed by other users. If a vulnerable version of jQuery (between 1.2 and 3.5.0) is used on a web interface of a device (like a web-based admin panel for the Ultimaker 3D printer), attackers can exploit XSS vulnerabilities to inject malicious scripts. [24]	5.4 (Medium) [24]	XSS amplifies the vulnerability by allowing attackers in a MitM attack to inject harmful scripts that can steal data, manipulate communication, and redirect users to malicious websites.

Building on these developments in machine learning security, the potential of LLMs in G-code analysis is an emerging area of interest. While originally designed for natural language processing, these models are now being explored for their potential in understanding and manipulating G-code. A recent paper from Jignasu et al. [27] highlighted the potential of LLMs to assist in debugging G-code, identify issues, and suggest optimizations. Extending on this work, in the paper, we explore the possibility of LLMs to identify malicious modifications to a G-code file, leading to better cyber-attack mitigation strategies.

3. Vulnerability Assessment

We first conducted a thorough vulnerability assessment of our 3D printer setup to identify and prioritize security weaknesses in our 3D printer system. Our approach is divided into the following three steps:

1. We performed a **literature review** focused on identifying vulnerabilities specific to 3D printing, especially MitM attacks and other common cyberattack vectors, which helped us understand the fundamental risks and attack vectors.
2. Next, we performed **network vulnerability scanning** using Nessus [28], a widely recognized vulnerability scanner, on the Ultimaker S5 printer to generate a fundamental vulnerability report, using a basic scan.
3. Based on the Nessus report, we identified vulnerabilities with entries in the **National Vulnerability Database (NVD)** [29] to verify them. The NVD, managed by the National Institute of Standards and Technology (NIST), provides standardized data about software security flaws. The database supplements vulnerability data with additional analysis, severity scoring, and impacts. Each vulnerability in the NVD is assigned a unique CVE (Common Vulnerabilities and Exposures) identifier, enabling consistent tracking and reference across the cybersecurity community. The database provides comprehensive information, including detailed vulnerability descriptions, affected software and versions, technical impact assessments, and severity ratings using the Common Vulnerability Scoring System (CVSS).

The CVSS provides standardized severity ratings from 0 to 10, categorizing vulnerabilities into several ranges: 0.0 (None), indicating no vulnerability detected; 0.1–3.9 (Low), minor vulnerabilities with minimal risk; 4.0–6.9 (Medium), moderate vulnerabilities requiring attention; 7.0–8.9 (High) severe vulnerabilities requiring immediate action; and 9.0–10.0 (Critical) signifying critical vulnerabilities that pose extreme risk. We searched the NVD specifically for high-severity vulnerabilities related to the Ultimaker S5 printer to ensure comprehensive coverage; we present our significant findings in Table 1. We found that cross-site request forgery (CSRF) and code injection as the most critical vulnerabilities of our 3D printer.

4. Man-in-the-Middle (MitM) Cyber Attack Simulation

The security analysis of the 3D printer software revealed several vulnerabilities within the printer firmware that could be exploited through MitM attacks [30]. Other researchers have also identified significant vulnerabilities, including uncontrolled format strings, path manipulation, command injection, and memory leaks [30]. Other studies [13, 31–35] also address the risks posed by MitM attacks.

Our vulnerability assessment identified code injection as a primary attack vector in MitM scenarios. Through these attacks, malicious actors exploit insecure network protocols to intercept communications between 3D printers and their connected devices, potentially compromising the integrity of printed objects. Previous research has demonstrated that attackers can successfully intercept printer model files and inject unauthorized print jobs into the workflow [36].

For our attack simulation, we assumed an adversary with network access and knowledge of the printer's IP address. The attack implementation utilized two primary tools:

- **Ettercap** [37]: For performing ARP spoofing and establishing the man-in-the-middle position
- **Tshark** [38]: For capturing and analyzing design files and G-code transmissions

The environment setup consists of four main components, as shown in Fig. 3. The network topology consists of a Windows-based user workstation running the Ultimaker Cura software, connected through a network router to an Ultimaker S5 3D printer. The router serves as the central network, providing WiFi to allow communication between the user workstation and the printer. The attacker system, running on a Linux platform, exploits the network system by poisoning the communication between the router and the target devices to enable the interception of all data sent to the printer.

We first perform a network reconnaissance using Tshark to monitor traffic between the Ultimaker Cura software and the S5 printer on the attacker's device. This analysis revealed that printer communications primarily occur through HTTP requests,

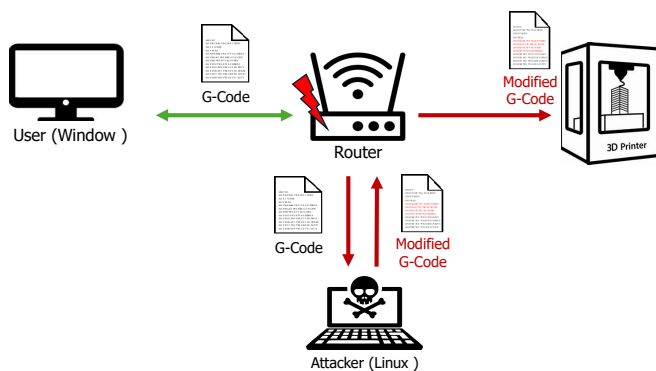


Fig. 3: System setup for executing a man-in-the-middle attack.

with print jobs transmitted in UFP (Ultimaker Format Package) format. These packages contain sliced G-code files, material specifications, and 3D model preview data [39]. Understanding these communication patterns proved crucial for identifying and targeting specific packet types, particularly those triggered by print operations and G-code transmissions.

To simulate the MitM attacks, we modified Ettercap's filtering capabilities by implementing a custom parsing function. These modifications enabled precise packet interception and manipulation between the Cura software and the printer. Specifically, we developed functionality to:

- Identify and parse print command signatures within packet payloads.
- Intercept and modify print request keywords.
- Inject modified print jobs with altered parameters.

Through this implementation, we successfully demonstrated the ability to intercept legitimate print requests, prevent authorized print jobs from reaching their destination, and inject unauthorized, modified print jobs into the workflow while maintaining the appearance of regular operation. This vulnerability highlights the critical need for enhanced security measures in additive manufacturing systems, particularly in scenarios where print job integrity is crucial for product quality and safety.

5. LLM-Based G-Code Modification Detection

In the previous section, we successfully demonstrated a MitM attack by modifying the G-code file sent to the printer. Older 3D printers do not use encryption, making the G-code susceptible to MitM attacks. This suggests that a successful attack mitigation strategy would be to validate the G-code file received by the 3D printer. G-code validation presents significant challenges due to its extensive length, often containing thousands of lines of code that make manual human review impractical and ineffective.

Since modern large language models (LLMs) have shown tremendous promise in generating and understanding programming languages, we propose to use LLMs to identify malicious modifications in G-code. Building upon prior research by Jignasu et al. [27] that identified ChatGPT and Claude as top performers in G-code error detection, we evaluated three state-of-the-art language models: ChatGPT-4o, ChatGPT o1-preview, and Claude Sonnet 3.5. Our evaluation employed a structured prompt methodology with two key components:

Prompt 1: We adapted the initial prompt from Jignasu et al. [27] to include comprehensive G-code command definitions and context. To address firmware-specific variations, we expanded this prompt to align with the G-code generated by the Ultimaker Cura software (Fig. 4), helping the LLM interpret commands that had previously caused potential confusion.

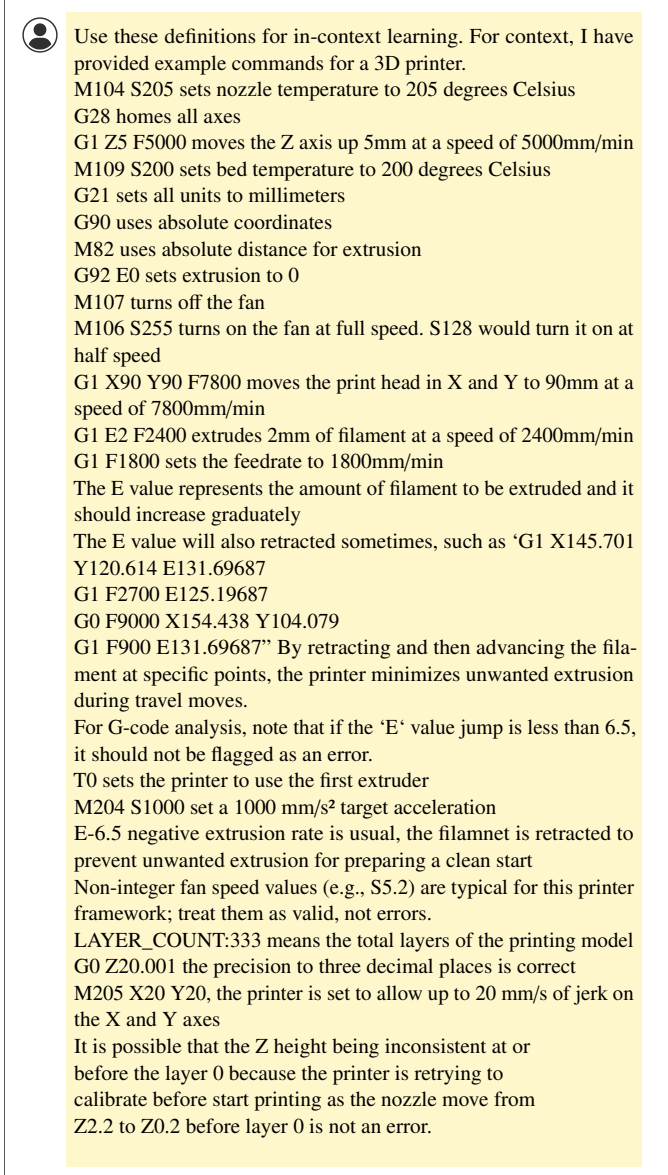


Fig. 4: Initial prompt to each LLM.

Prompt 2: Taken from the same study, this prompt directly asks the model to locate errors with the statement, “Hey, I think there’s erroneous lines in this G-Code file, could you help me find it?,” as shown in Fig. 5.

To systematically assess the ability of the LLMs, we use a **Cone** model and a **Square Pyramid** model shown in Fig. 6. The cone model was sliced into 26 layers using Cura, producing a G-code file with 1704 lines, while the square pyramid was sliced into 31 layers, resulting in 1676 lines. Our method follows a structured approach for all dialogues, designed to optimize the LLM’s ability to process and analyze the G-code file. The different steps in our prompt approach are:

1. **Consistent Prompt Initiation:** Each dialogue begins with the same foundational prompt, offering key G-code defini-

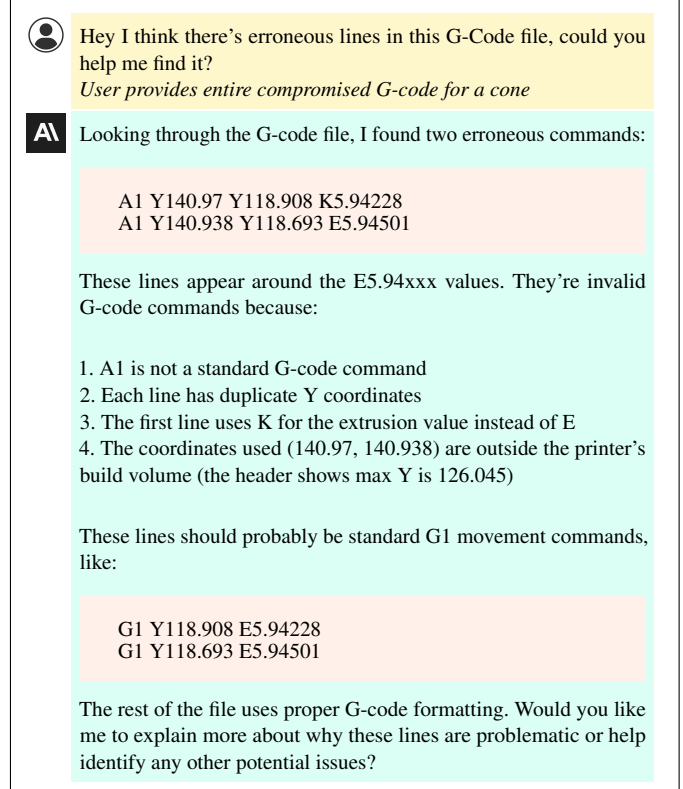


Fig. 5: Prompting Claude-2 to identify errors in the G-Code.

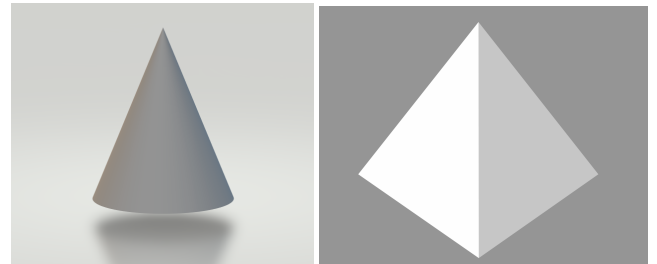


Fig. 6: Cone and Pyramid models used for testing G-code modifications.

tions and context relevant to Ultimaker Cura’s format to ensure consistent in-context learning

2. **Full G-Code Input:** Given the small file size (1704 lines or 1676 lines), we provide the entire G-code file as input to each model, enabling access to all code layers and commands in a single session. After the model accurately detected errors without false positives, we repeated the process twice more to verify its stability.
3. **Guided Prompting:** If errors were missed, additional guidance specified the error type.

In our evaluation, we tested nine attack scenarios for the LLMs to detect. Initially, we experimented with injecting, duplicating, and removing G-code, but these modifications altered the file size, making them detectable by methods such as checksums. To address this, we refined our approach to modify the G-code

Table 2: Cone: Performance comparison of LLMs in detecting G-code modifications. Color coding represents the outcomes that were classified as:  Error detected in the initial prompt without false positive results (1 point),  Error identified after guidance (0.5 point), and  Error missed despite guidance (0 point).

Capability	Claude	GPT o1-preview	GPT-4o
Inject random information at a specific location	1	0.5	0.5
Inject random information at two positions	0.5	0.5	0.5
Duplicate two layers	0.5	0.5	0.5
Remove two layers	0.5	0.5	0.5
Remove 15 layers	0.5	0.5	0.5
Modify Z-axis height	0.5	0	0
Modify E value	1	0.5	0.5
Replace two lines of code	1	1	0.5
Replace X and G commands with other characters	1	0	0
Overall Performance	6.5/9 $\approx$ 72%	4/9 $\approx$ 44%	3.5/9 $\approx$ 39%

Table 3: Pyramid: Performance comparison of LLMs in detecting G-code modifications. Color coding represents the outcomes that were classified as:  Error detected in the initial prompt without false positive results (1 point),  Error identified after guidance (0.5 point), and  Error missed despite guidance (0 point).

Capability	Claude	GPT o1-preview	GPT-4o
Inject random information at a specific location	1	0.5	0.5
Inject random information at two positions	1	1	0
Duplicate two layers	0.5	1	0.5
Remove two layers	0.5	0.5	0.5
Remove 15 layers	0.5	0.5	0.5
Modify Z-axis height	0.5	0.5	0.5
Modify E value	1	0.5	0
Replace two lines of code	1	1	0.5
Replace X and G commands with other characters	1	1	0
Overall Performance	7/9 $\approx$ 78%	6.5/9 $\approx$ 72%	2.5/9 $\approx$ 28%

without affecting file size, focusing on changes like adjusting Z or E values and replacing code with alternative characters.

The performance of each LLM model varied across different types of G-code modifications for the Cone (Table 2) and Pyramid (Table 3) CAD models. Claude excelled in initial detection, particularly in identifying random injections, E-value modifications, and code replacements.

In the analysis of G-Code generation for the Cone model, ChatGPT o1-preview and ChatGPT 4o performed similarly overall, though o1-preview uniquely detected code replacement issues—demonstrating its ability to spot formatting errors that ChatGPT 4o missed. Both models required additional guidance, and some outputs were flagged as "yellow" for partial correctness.

Notably, while Claude and ChatGPT o1-preview maintained high-quality results for the Pyramid model—a contrast to the Cone model evaluation—ChatGPT 4o's performance declined on the Pyramid compared to the Cone, even though the Cone is generally more geometrically complex. This suggests that ChatGPT 4o's performance does not correlate linearly with geometric complexity, highlighting the need for comprehensive

testing across diverse shapes to accurately assess AI capabilities in G-Code generation.

In the analysis of G-Code generation for the Cone model, ChatGPT o1-preview showed limitations in error detection when characters were replaced in the G-code. When X and G commands were substituted with other characters, the model only identified one of two actual error lines while also generating false positives, as shown in Fig. B.8, which is the reason why it is flagged as "red." Although performance improved with the Pyramid model, the error identification remained only partially accurate.

Besides that, the ChatGPT models incorrectly identified non-existent code lines as errors, as demonstrated in Fig. B.9, where it referenced a second line of code that was not present in the provided file. In the dialog shown in Fig. B.10, the ChatGPT 4o identified the error lines with guided assistance but also produced a partially correct response. While ChatGPT o1-preview successfully identified one error line with a modified Z value, it missed another line with a similar modification, as shown in Fig. B.11.

Table 4: Cone: Confusion matrix for different LLMs.

Claude Sonnet 3.5	Actually Positive	Actually Negative
Predicted Positive	90%	60 %
Predicted Negative	10%	40%
GPT o1-preview	Actually Positive	Actually Negative
Predicted Positive	75%	0 %
Predicted Negative	25%	100%
GPT-4o	Actually Positive	Actually Negative
Predicted Positive	15%	70 %
Predicted Negative	85%	30%

Table 5: Pyramid: Confusion matrix for different LLMs.

Claude Sonnet 3.5	Actually Positive	Actually Negative
Predicted Positive	100%	0 %
Predicted Negative	0%	100%
GPT o1-preview	Actually Positive	Actually Negative
Predicted Positive	100%	40 %
Predicted Negative	0%	60%
GPT-4o	Actually Positive	Actually Negative
Predicted Positive	20%	10 %
Predicted Negative	80%	90%

We calculated confusion matrices (Table 4 and Table 5) for different LLMs to numerically evaluate and compare their capabilities in G-code error detection. The confusion matrix organizes each model's predicted classifications along the rows and actual (ground truth) labels along the columns. We defined positive cases as instances where G-code contains errors and negative cases as unmodified G-code sequences. For each LLM model under evaluation, the matrix comprises four key components that quantify detection performance:

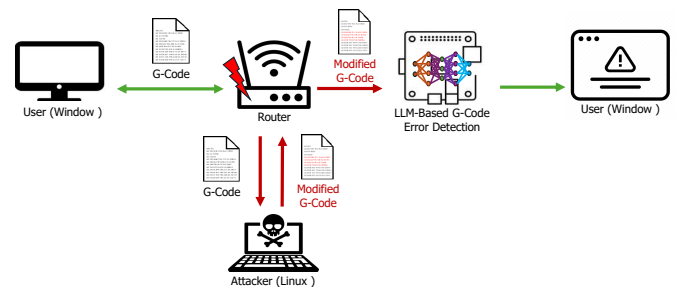
- True Positives (Top left): These represent instances where the LLM correctly identified modified or erroneous G-code, demonstrating effective error recognition.
- False Positives (Top right): These cases indicate where the LLM incorrectly flagged unmodified G-code as erroneous.
- False Negatives (Bottom left): These instances represent modified G-code that the LLM failed to detect.
- True Negatives (Bottom right): These cases demonstrate the LLM's ability to identify unmodified G-code sequences correctly.

This structured evaluation framework enables a comprehensive comparison of different LLM models' G-code analysis capabilities through standard performance metrics. These metrics provide quantitative insights into each model's effectiveness in detecting G-code errors. We have included the complete data for each model in the Appendix.

6. Conclusions and Future Work

In summary, our research reveals significant vulnerabilities in current AM systems, especially in G-code transmission and verification. Through practical MitM attack implementations, we demonstrated the potential manipulation of G-code instructions and their impact on print integrity. Lastly, we explored and validated the capabilities of LLMs in analyzing G-code content for anomalies and potential threats, revealing a potential new direction for intelligent security measures in additive manufacturing.

While encryption and other traditional defense methods can mitigate MitM attacks, our LLM-based detection strategy offers an additional, lightweight security layer. This approach is particularly crucial, as research indicates nearly half of the manufacturing industries experienced cyberattacks in 2018 [40]. Despite

**Fig. 7:** Workflow for LLM-based detection of G-Code modifications.

widespread awareness of cybersecurity risks, many manufacturers still lack appropriate security controls and standards. Our solution, demonstrated in Fig. 7, can run on resource-constrained devices such as Raspberry Pi, making it particularly valuable for legacy systems and older firmware versions that often lack modern security features.

Another future direction is to compare the LLM-based detection approach with traditional cyberattack detection methods. Specifically, we can evaluate LLM-based detection against rule-based anomaly detection methods (e.g., syntax verification, checksum validation) and machine learning-based classifiers (e.g., support vector machines, autoencoders) in future studies. We believe incorporating LLMs for detecting modified G-codes is a practical application of modern AI tools in cybermanufacturing.

Acknowledgements

This material is based on research sponsored by Office of the Under Secretary of Defense for Research and Engineering, Strategic Technology Protection and Exploitation, and Defense Manufacturing Science and Technology Program under agreement number W15QKN-19-3-0003. The U.S. Government is authorized to reproduce and distribute reprints for governmental purposes notwithstanding any copyright notation thereon. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the U.S. Government. This work was also partly supported by the National Science Foundation under grant number CMMI-2347623.

References

- [1] G. V. Research, 3D printing industry analysis, 2023. URL: <https://www.grandviewresearch.com/industry-analysis/3d-printing-industry-analysis>, accessed: 2024-07-11.
- [2] National Aeronautics and Space Administration, Additive manufacturing subtracts from rocket build time, 2023. URL: https://spinoff.nasa.gov/Additive_Manufacturing_Subtracts_from_Rocket_Build_Time, accessed on [Insert Date of Access].
- [3] M. Mehrpouya, A. Dehghanghadikolaie, B. Fotovvati, A. Vosooghnia, S. S. Emamian, A. Gisario, The potential of additive manufacturing in the smart factory industrial 4.0: A review, *Applied Sciences* 9 (2019) 3865.
- [4] M. Javaid, A. Haleem, Current status and applications of additive manufacturing in dentistry: A literature-based review, *Journal of oral biology and craniofacial research* 9 (2019) 179–185.
- [5] C. Culmone, G. Smit, P. Breedveld, Additive manufacturing of medical instruments: A state-of-the-art review, *Additive manufacturing* 27 (2019) 461–473.
- [6] A. S. Salles, D. E. Gyi, Delivering personalised insoles to the high street using additive manufacturing, *International Journal of Computer Integrated Manufacturing* 26 (2013) 386–400.
- [7] B. Blakey-Milner, P. Gradl, G. Snedden, M. Brooks, J. Pitot, E. Lopez, M. Leary, F. Berto, A. Du Plessis, Metal additive manufacturing in aerospace: A review, *Materials & Design* 209 (2021) 110008.
- [8] J. C. Vasco, Additive manufacturing for the automotive industry, in: *Additive Manufacturing*, Elsevier, 2021, pp. 505–530.
- [9] M. Yampolskiy, A. Skjellum, M. Kretzschmar, R. A. Overfelt, K. R. Sloan, A. Yasinsac, Using 3d printers as weapons, *International Journal of Critical Infrastructure Protection* 14 (2016) 58–71.
- [10] F. W. Baumann, D. Roller, Additive manufacturing, cloud-based 3d printing and associated services—overview, *Journal of Manufacturing and Materials Processing* 1 (2017) 15.
- [11] L. D. Sturm, C. B. Williams, J. A. Camelio, J. White, R. Parker, Cyber-physical vulnerabilities in additive manufacturing systems: A case study attack on the .stl file with human subjects, *Journal of Manufacturing Systems* 44 (2017) 154–164. URL: <https://www.sciencedirect.com/science/article/pii/S0278612517300961>. doi:<https://doi.org/10.1016/j.jmsy.2017.05.007>.
- [12] G. Sriram, S. Rambabu, Importance of cybersecurity in additive manufacturing in the era of industry 4.0, *Journal ID* 2940 (2024) 6442.
- [13] M. McCormack, S. Chandrasekaran, G. Liu, T. Yu, S. D. Wolf, V. Sekar, security analysis of networked 3d printers, in: 2020 IEEE Security and Privacy Workshops (SPW), IEEE, 2020, pp. 118–125.
- [14] S.-Y. Yu, A. V. Malawade, S. R. Chhetri, M. A. Al Faruque, Sabotage attack detection for additive manufacturing systems, *IEEE Access* 8 (2020) 27218–27231.
- [15] J. Rossel, V. Mladenov, J. Somorovsky, Security analysis of the 3mf data format, in: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, 2023, pp. 179–194.
- [16] H. Rais, Material extrusion-based additive manufacturing: G-code and firmware attacks and Defense frameworks, Ph.D. thesis, Virginia Commonwealth University, 2023.
- [17] Xometry, What is g-code?, <https://www.xometry.com/resources/machining/what-is-g-code/>, 2024. Accessed: 2024-11-11.
- [18] Ultimaker, Ultimaker cura, 2011–2023. URL: <https://ultimaker.com/software/ultimaker-cura/>.
- [19] KirstenS, Cross site request forgery (csrf), n.d. URL: <https://owasp.org/www-community/attacks/csrf>, accessed: [08/10/2023].
- [20] N. V. Database, Nvd-cve-2021-34086, 2021. URL: <https://nvd.nist.gov/vuln/detail/CVE-2021-34086>, accessed: 2024-11-04.
- [21] National Vulnerability Database, NVD-CVE-2024-8374, 2024. URL: <https://nvd.nist.gov/vuln/detail/CVE-2024-8374>, accessed: 2024-11-04.
- [22] MITRE, Cwe-94: Improper control of generation of code ('code injection'), 2024. URL: <https://cwe.mitre.org/data/definitions/94.html>, accessed: 2024-11-11.
- [23] N. V. Database, Nvd-cve-2021-34087, 2022. URL: <https://nvd.nist.gov/vuln/detail/CVE-2021-34087>, accessed: 2024-11-04.
- [24] National Vulnerability Database, NVD - CVE-2024-10385, 2024. URL: <https://nvd.nist.gov/vuln/detail/CVE-2024-10385>, accessed: 2024-11-04.
- [25] C. Beckwith, H. S. Naicker, S. Mehta, V. R. Udupa, N. T. Nim, V. Gadre, H. Pearce, G. Mac, N. Gupta, Needle in a haystack: Detecting subtle malicious edits to additive manufacturing g-code files, *IEEE Embedded Systems Letters* 14 (2021) 111–114.
- [26] Q. Liu, X. Y. Lee, S. Sarkar, A study of natural robustness of deep reinforcement learning algorithms towards adversarial perturbations, *AI Open* 5 (2024) 126–141.
- [27] A. Ignasus, K. Marshall, B. Ganapathysubramanian, A. Balu, C. Hegde, A. Krishnamurthy, Evaluating large language models for g-code debugging, manipulation, and comprehension, in: 2024 IEEE LLM Aided Design Workshop (LAD), IEEE, 2024, pp. 1–5.
- [28] I. Tenable, Nessus Vulnerability Scanner: Network Security Solution, 2024. URL: <https://www.tenable.com/products/nessus>, accessed: 2024-11-14.
- [29] National Institute of Standards and Technology, Vulnerability metrics, 2024. URL: <https://nvd.nist.gov/vuln-metrics>, accessed: 2024-11-14.
- [30] S. Moore, P. Armstrong, T. McDonald, M. Yampolskiy, Vulnerability analysis of desktop 3d printer software, in: 2016 Resilience Week (RWS), IEEE, 2016, pp. 46–51.
- [31] Q. Do, B. Martini, K.-K. R. Choo, A data exfiltration and remote exploitation attack on consumer 3d printers, *IEEE Transactions on Information Forensics and Security* 11 (2016) 2174–2186. doi:[10.1109/TIFS.2016.2578285](https://doi.org/10.1109/TIFS.2016.2578285).
- [32] S. Belikovetsky, Y. A. Solewicz, M. Yampolskiy, J. Toh, Y. Ellovici, Digital audio signature for 3d printing integrity, *IEEE Transactions on Information Forensics and Security* 14 (2019) 1127–1141. doi:[10.1109/TIFS.2018.2851584](https://doi.org/10.1109/TIFS.2018.2851584).

- [33] S. E. Zeltmann, N. Gupta, N. G. Tsoutsos, M. Maniatakos, J. Rajendran, R. Karri, Manufacturing and security challenges in 3d printing, *Jom* 68 (2016) 1872–1881.
- [34] M. Yampolskiy, L. Schutzle, U. Vaidya, A. Yasinsac, Security challenges of additive manufacturing with metals and alloys, in: *Critical Infrastructure Protection IX: 9th IFIP 11.10 International Conference, ICCIP 2015, Arlington, VA, USA, March 16–18, 2015, Revised Selected Papers 9*, Springer, 2015, pp. 169–183.
- [35] M. Yampolskiy, T. R. Andel, J. T. McDonald, W. B. Glisson, A. Yasinsac, Intellectual property protection in additive layer manufacturing: Requirements for secure outsourcing, in: *Proceedings of the 4th Program Protection and Reverse Engineering Workshop*, 2014, pp. 1–9.
- [36] Q. Do, B. Martini, K.-K. R. Choo, A data exfiltration and remote exploitation attack on consumer 3d printers, *IEEE Transactions on Information Forensics and Security* 11 (2016) 2174–2186.
- [37] Ettercap Project, Ettercap, 2024. URL: <https://www.ettercap-project.org/about.html>.
- [38] R. Jacobs, Tshark, 2024. URL: <https://tshark.dev/>.
- [39] Ultimaker Support, Why is my gcode not recognized on the UltiMaker 2+ Connect?, 2024. URL: <https://support.ultimaker.com/hc/en-us/articles/19611498749841>.
- [40] J. Evans, Be prepared for the security challenges of industry 4.0, 2018. URL: <https://www.orange-business.com/en/blogs/be-prepared-security-challenges-industry-40>, accessed: 2024-11-11.

Appendix A. Complete Evaluation Data

Below are the raw data for Table 4 and Table 5, representing the model unmodified G-code and two code modification scenarios with erroneous lines. These scenarios were selected because the model performed better in them, as observed in Table 2 and Table 3. The color coding applies to all tables: Error detected in the initial prompt without false positives (1 point), Error identified after guidance (0.5 point), and Error missed despite guidance (0 points).

Table A.6: Performance of LLMs on prompt repetition with the Cone geometry.

Unmodified G-code	Claude Sonnet 3.5	GPT o1-preview	GPT-4o
Trial 1	1	1	0
Trial 2	0.5	1	0
Trial 3	0	1	0.5
Trial 4	1	1	0.5
Trial 5	0.5	1	0.5
Replace Two Lines of G-code	Claude Sonnet 3.5	GPT o1-preview	GPT-4o
Trial 1	1	1	0
Trial 2	1	1	0.5
Trial 3	1	1	0.5
Trial 4	1	1	0
Trial 5	1	1	0.5
Modify E value in G-code	Claude Sonnet 3.5	GPT o1-preview	GPT-4o
Trial 1	0.5	0.5	0
Trial 2	0.5	0.5	0
Trial 3	1	0.5	0
Trial 4	1	0.5	0
Trial 5	1	0.5	0

Table A.7: Performance of LLMs on prompt repetition with the Pyramid geometry.

No. of Test	Claude Sonnet 3.5	GPT o1-preview	GPT-4o
1	1	0.5	1
2	1	1	1
3	1	0.5	1
4	1	0.5	1
5	1	0.5	0.5
Replace two lines of G-code	Claude Sonnet 3.5	GPT o1-preview	GPT-4o
1	1	1	0.5
2	1	1	0.5
3	1	1	0
4	1	1	1
5	1	1	0
Inject random information at two locations	Claude Sonnet 3.5	GPT o1-preview	GPT-4o
1	1	1	0
2	1	1	0
3	1	1	0
4	1	1	0
5	1	1	0

Appendix B. Detailed LLM Chats

 Hey I think there's erroneous lines in this G-Code file, could you help me find it?
User provides entire compromised G-code for a cone

 Yes, I can help you find the erroneous line in your G-code file. After reviewing the code, I found an error on this line:

G1 F1035 X145.304 X115.139 E3.88586

Issue: The command contains two X parameters, which is invalid in G-code syntax. Each movement command (G1) should have at most one X, Y, Z, or E parameter for each axis or function.

Solution: The second X should likely be a Y to specify movement in the Y-axis. The corrected line should be:

G1 F1035 X145.304 Y115.139 E3.88586

Explanation: This correction ensures that the printer moves to the correct Y-coordinate (Y115.139) while extruding the specified amount of filament (E3.88586).

 No, this is no the error. Can you find the line might not follow the format?

 Provided the same response as previous response.

 Can you find all the lines that are not follow the format ?

 Certainly! After thoroughly reviewing your G-code file, I have identified all the lines that do not follow the standard G-code format. Below is a list of these lines, along with explanations of the issues and suggested corrections.

1. Lines with Duplicate 'X' Parameters
....
2. Lines with Invalid 'A1' Commands

A1 Y140.97 Y118.908 K5.94228

3. Line with Missing Axis Parameter
....
4. Lines with Missing 'G1' Command
....
5. Lines with Multiple Identical Parameters
....

Fig. B.8: ChatGPT o1-preview failed to identify the code that is modified.

 Hey I think there's erroneous lines in this G-Code file, could you help me find it?
User provides entire compromised G-code for a cone

 I'll go through the file based on the definitions you've shared and note any deviations or potential issues. Here are some of the highlights that could require correction or closer inspection: 5. Command Errors:

There's a non-standard command line with A1 (e.g., A1 Y140.938 Y118.693 E5.94501), which doesn't follow G-code syntax. This line might be an error or misplaced and should be corrected or removed.
....

 There are more errors like a non-standard command line with A1 (e.g., A1 Y140.938 Y118.693 E5.94501).

 Got it! I'll systematically search for any instances where non-standard commands like A1 or other unusual entries deviate from the expected G-code syntax. I'll compile a list of these for you to review.

Give me a moment to check thoroughly.

Here are the non-standard command lines identified in your G-code file:

A1 Y140.938 Y118.693 E5.94501
A1 Y140.900 Y118.640 E5.96501

These lines use A1 instead of the standard G-code commands (G, M, or T). They might have been added by mistake or due to a formatting issue in the code generation. It would be best to remove or replace these with standard commands if they were intended for movement or extrusion.

Let me know if you'd like to search for more specific issues or correct these entries.

 There are more errors like a non-standard command line with A1 (e.g., A1 Y140.938 Y118.693 E5.94501). Could you find them?

 To identify all instances of non-standard commands like A1 in the provided G-code, I will process the file and locate such commands. Let me extract and analyze the code.

It seems I couldn't find any additional occurrences of the non-standard command A1 in the provided G-code. If you believe there are more instances, please double-check the file or provide additional details about the patterns or structures you're expecting to locate.

Fig. B.9: ChatGPT 4o: Misidentifying Nonexistent Lines as Errors

 Hey I think there's erroneous lines in this G-Code file, could you help me find it?
User provides entire compromised G-code for a cone

 The G-code file begins with header information specific to the printing setup, including the software used, machine name, and temperature settings. I'll now analyze the rest of the file according to your criteria to identify any potential errors, such as E-value jumps over 6.5, non-standard fan speeds, or issues with Z-height calibration. I'll examine the remaining lines and highlight any that don't align with your printer's parameters.

The analysis identified several lines with potential issues based on your criteria:

E-value Jumps: Some lines have extrusion (E-value) jumps exceeding the 6.5 threshold, indicating possible excessive material extrusion between commands:

```
G1 F2700 E-6.5
G1 F2700 E0
G1 X142.27 Y119.503 E20.85703
G1 F436 X142.45 Y119.196 E6.9022
G1 F2700 E2.9064
```

If these specific values are intentional, you might adjust the E-jump threshold. Let me know if you want to review other potential issues in detail or further refine the analysis.

Fig. B.10: ChatGPT 4o: Detecting modified E values in the G-code lines

 Certainly! After reviewing your G-code file against the provided definitions, I believe the erroneous line is:

```
G1 F2700 E2.9064
```

 This is correct. Check if there are any missing code or layers/ injected some duplicated code or random information or big jump of height or layers.

 Certainly! After a thorough analysis of your G-code file, I have identified another issue related to a sudden and significant jump in the Z-axis height, which could potentially cause printing errors. Here's a detailed breakdown: Issue: Big Jump in Z Height at Layer 6 Observation:

- Layer 5 Ends at Z = 0.95 mm: At the end of Layer 5, the Z-height is at 0.95 mm.

```
;LAYER:5
G0 F600 X142.322 Y118.654 Z0.95
```

- Unexpected Jump to Z = 10 mm: Before starting Layer 6, the Z-height suddenly jumps to 10 mm.

```
;MESH:NONMESH
G0 F600 X141.831 Y118.777 Z10
G0 F9000 X141.029 Y118.396
;TIME_ELAPSED:36.794405
;LAYER:6
```

- Layer 6 Starts at Z = 1.4 mm: In Layer 7, the Z-height drops back to 1.4 mm.

```
;LAYER:7
G0 F600 X141.784 Y118.747 Z1.4
```

...

Fig. B.11: ChatGPT o1-preview: Detecting Modified Z Value Lines with Guiding Prompts